

GitHub: Repositories, Branches, Pull Requests, and Issues

A step-by-step course for IT undergraduates

From an idea to a reviewed change in main

GIT

GITHUB

TEAMWORK

```
git switch -c feature/login  
git add . && git commit -m "feat: add login"  
git push -u origin feature/login
```

Learning outcomes

What students will be able to do after the class

01

Explain

Git, GitHub, repository, commit, and remote

02

Work

with branches without changing main directly

03

Collaborate

through Pull Requests, reviews, and merges

04

Plan

work with Issues, labels, and milestones

Git and GitHub solve different problems

Local history and a cloud platform complement each other

Git

Distributed version control system

- stores project snapshots as commits
- creates branches and combines history
- works locally without internet access
- runs git commands

Local

GitHub

Platform for Git repositories

- hosts a remote copy
- manages team access
- supports Pull Requests and Issues
- runs checks and automation

Remote

Core terminology

One model connects local work with GitHub

Repository

project files plus version history

Commit

a recorded snapshot of changes

Branch

a movable pointer to a commit chain

Remote

a reference to a remote repository

origin

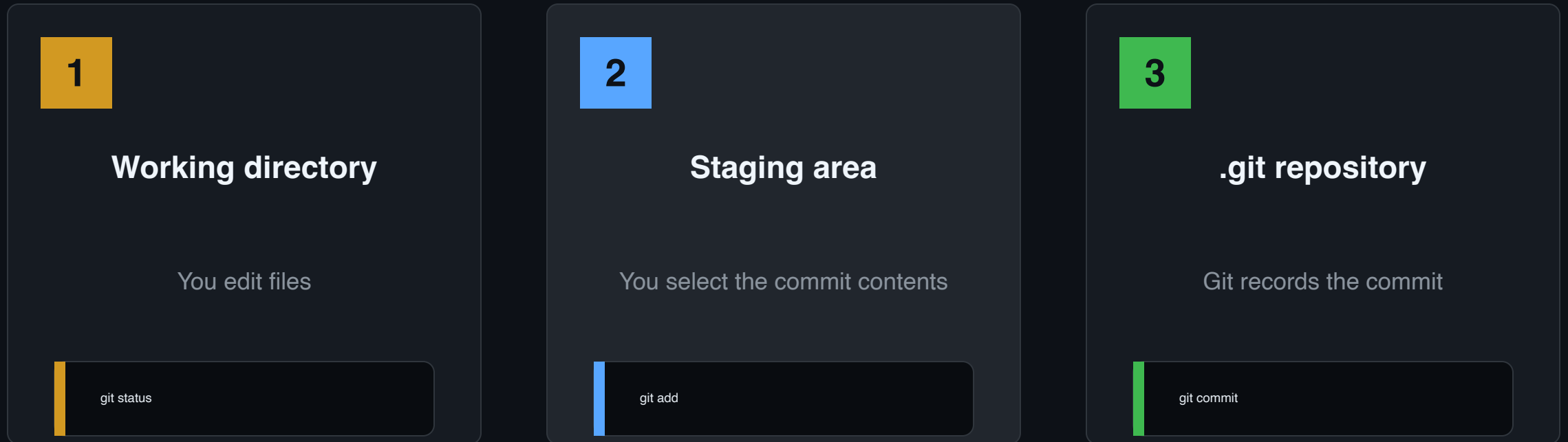
the conventional primary remote name

main

a common default branch name

Three areas of local work

A commit contains only staged changes



A changed file will not enter the next commit until you stage it with git add.

Initial Git configuration

Name and email become the author identity of new commits

```
git --version
```

```
git config --global user.name "Farikha Nauruzbaeva"  
git config --global user.email "student@example.com"
```

```
git config --global init.defaultBranch main  
git config --list
```

Option meanings

--global
setting for the current user

user.name / user.email
commit author, not GitHub login

init.defaultBranch
branch name created by git init

Two ways to obtain a repository

git init creates a new one; git clone copies an existing one

New local project

```
mkdir student-task-manager  
cd student-task-manager  
git init
```

Creates a hidden .git directory.
The history is initially empty.

Project from GitHub

```
git clone https://github.com/user/student-task-  
manager.git  
cd student-task-manager
```

Copies files, history, and remote references.

Creating a repository on GitHub

A minimal structure makes the project understandable and safer

README.md

purpose, setup, and examples

.gitignore

files that Git should not track

LICENSE

terms for using the code

src/ and tests/

source code and tests

```
# .gitignore
node_modules/
.env
dist/
*.log
```

Never commit secrets, even to a private repository.

First commit and publication

Inspect, stage, commit, then push

```
echo "# Student Task Manager" > README.md
```

```
git status
```

```
git add README.md
```

```
git commit -m "docs: add project README"
```

```
git remote add origin https://github.com/user/student-task-manager.git
```

```
git push -u origin main
```

Options

-m

commit message text

-u / --set-upstream

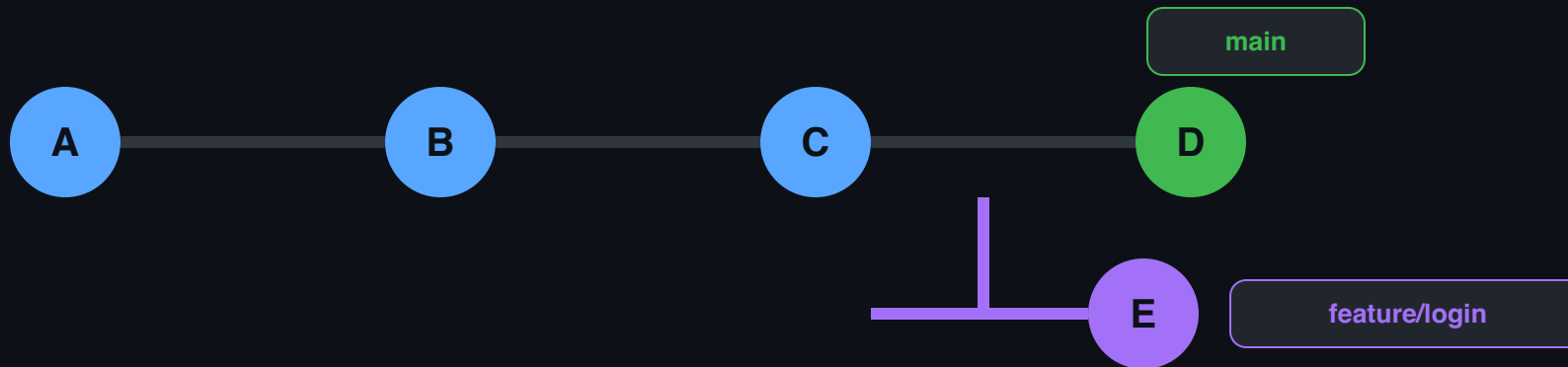
link the local branch to origin/
main

Afterward, this is enough:

git push

A branch isolates a task

Each feature or fix gets a separate line of work



```
git switch -c feature/login  
# -c = create: create and switch immediately
```

```
git branch  
git status
```

Feature-branch work cycle

Small commits are easier to review, explain, and undo

```
git switch -c feature/login
```

```
# edit src/auth.js and tests/auth.test.js
```

```
git diff
```

```
git status
```

```
git add src/auth.js tests/auth.test.js
```

```
git diff --staged
```

```
git commit -m "feat: add login validation"
```

```
git push -u origin feature/login
```

A good commit

One logical purpose

A clear message

Code with related tests

No temporary files or secrets

feat: action + object

Synchronizing with the remote repository

fetch downloads information; pull also integrates changes

`git fetch origin`

Download new commits and update origin/*
Current working files remain unchanged

`git pull --ff-only`

Download and advance only when history has not diverged
The command stops if branches diverged

`git push`

Send local commits to the upstream branch

A useful start-of-work habit: `git switch main && git pull --ff-only`

A Pull Request proposes combining changes

A PR turns a set of commits into a reviewable change

1

Push

the branch appears
on GitHub

2

Open PR

base: main,
compare: feature/
login

3

Review

comments and
change requests

4

Checks

tests, linter, and
build

5

Merge

the change enters
main

base receives changes

compare/head contains the proposed changes

Creating a Pull Request step by step

Examples using GitHub CLI and the web interface

```
git push -u origin feature/login
```

```
# GitHub CLI  
gh pr create \  
  --base main \  
  --head feature/login \  
  --title "Add login validation" \  
  --body "Closes #12"
```

Using the GitHub website

1. Open the repository
2. Select Compare & pull request
3. Verify base and compare
4. Explain what changed and why
5. Add reviewers and create the PR

Review checks code and transfers knowledge

The author responds with new commits in the same branch

Author

explains context
keeps the PR small
responds to feedback

Reviewer

checks correctness
evaluates tests and readability
explains risks

Checks

automatically run tests,
linting, builds, and
security checks

```
# correction after review
git add .
git commit -m "fix: handle empty email"
git push
```

Three Pull Request merge strategies

The choice changes history shape, not the resulting files

Merge commit

Keeps all commits and creates a separate merge commit

Useful when branch structure matters

Squash and merge

Combines all PR commits into one commit

A clean history for small PRs

Rebase and merge

Replays commits on top of base without a merge commit

Linear history with each commit retained

A conflict occurs when changes are incompatible

Git stops and asks a person to choose the final result

```
git switch feature/login
git fetch origin
git merge origin/main

# Git reports a conflict
<<<<<< HEAD
return validateEmail(email);
=====
return validateCredentials(email, password);
>>>>>> origin/main
```

Resolution

1. Open the conflicted file
2. Select or combine the correct code
3. Remove <<<<, ==, >>>> markers
4. Run the tests
5. git add <file>
6. git commit && git push

An Issue describes work or a problem

A good Issue provides context and a verifiable outcome

Issue #12: Validate the login form

Problem

The form submits a request with an empty email.

Expected behavior

Show the message “Enter an email address.”

Acceptance criteria

- ☐ an empty field blocks submission
- ☐ a valid email passes validation
- ☐ unit tests are included

Metadata

Assignee

person responsible

Labels

type, priority, area

Milestone

release or phase

Project

board and status

Issues and Pull Requests connect plans to code

One workflow preserves traceability

Issue #12

What is needed and why

Branch

issue-12-login-validation

Commits

Small implementation steps

PR #18

Review, checks, discussion

Closed

Issue closes after merge

```
git switch -c issue-12-login-validation  
# PR description: Closes #12
```

Team workflow

A repeatable process reduces conflicts and lost work

01

Create an Issue

02

Update main

03

Create a branch

04

Make commits

05

Push and open PR

06

Review and checks

07

Merge and delete branch

```
git switch main && git pull --ff-only  
git switch -c issue-12-login-validation
```

Common mistakes and safe responses

Inspect the state first, then make the correction

Committed on main

move the work to a branch before push

`git switch -c fix/move-work`

Staged an extra file

unstage it while keeping the file

`git restore --staged file`

Undo a published commit

create a new inverse commit

`git revert <hash>`

Push rejected

download and integrate remote changes

`git fetch / git pull`

Secret in history

revoke the secret and clean history

rotate/revoke first

Command cheat sheet

A minimal set for daily work

State

```
git status
git diff
git log --oneline --graph
```

Branches

```
git branch
git switch <name>
git switch -c <new>
```

Changes

```
git add <file>
git commit -m "message"
git restore --staged <file>
```

Remote

```
git fetch origin
git pull --ff-only
git push -u origin <branch>
```